

DSA: Class&Lab

- ARRAYS

****print elements of an array:**

Ans:

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, i;
```

```
    printf("Enter the number of elements in the array: ");
```

```
    scanf("%d", &n);
```

```
    int arr[n];
```

```
    printf("Enter the elements of the array:\n");
```

```
    for (i = 0; i < n; i++) {
```

```
        scanf("%d", &arr[i]);
```

```
    }
```

```
printf("The elements of the array are:\n");  
for (i = 0; i < n; i++) {  
    printf("%d ", arr[i]);  
}  
  
return 0;  
}
```

**** find position(index) of an element**

Ans:

```
#include <stdio.h>
```

```
void findIndex(int arr[], int n, int key) {  
    for (int i = 0; i < n; i++) {  
        if (arr[i] == key) {  
            printf("Element %d is found at index %d.\n", key, i);  
            return;  
        }  
    }  
    printf("Element %d is not found in the array.\n", key);  
}
```

```
int main() {  
    int arr[] = {10, 20, 30, 40, 50};  
    int n = sizeof(arr) / sizeof(arr[0]);  
    int key = 30;  
  
    findIndex(arr, n, key);  
  
    return 0;  
}
```

1) Insertion, Deletion Operations

Ans:

```
#include <stdio.h>  
  
void insertElement(int arr[], int *size, int element, int  
position) {  
    if (position < 0 || position > *size) {  
        printf("Invalid position!\n");  
        return;  
    }  
}
```

```
    for (int i = *size; i > position; i--) {  
        arr[i] = arr[i - 1];  
    }  
    arr[position] = element;  
    (*size)++;  
}
```

```
int findElement(int arr[], int size, int element) {  
    for (int i = 0; i < size; i++) {  
        if (arr[i] == element) {  
            return i;  
        }  
    }  
    return -1;  
}
```

```
void deleteElement(int arr[], int *size, int position) {  
    if (position < 0 || position >= *size) {  
        printf("Invalid position!\n");  
        return;  
    }  
}
```

```
    for (int i = position; i < *size - 1; i++) {  
        arr[i] = arr[i + 1];  
    }  
    (*size)--;  
}
```

```
void displayArray(int arr[], int size) {  
    for (int i = 0; i < size; i++) {  
        printf("%d ", arr[i]);  
    }  
    printf("\n");  
}
```

```
int main() {  
    int arr[100];  
    int size = 0;  
    int choice, element, position;  
  
    while (1) {  
        printf("\nArray Operations:\n");  
        printf("1. Insert Element\n");
```

```
printf("2. Find Element\n");  
printf("3. Delete Element\n");  
printf("4. Display Array\n");  
printf("5. Exit\n");  
printf("Enter your choice: ");  
scanf("%d", &choice);
```

```
switch (choice) {
```

```
    case 1:
```

```
        printf("Enter element to insert: ");  
        scanf("%d", &element);  
        printf("Enter position to insert (0 to %d): ", size);  
        scanf("%d", &position);  
        insertElement(arr, &size, element, position);  
        break;
```

```
    case 2:
```

```
        printf("Enter element to find: ");  
        scanf("%d", &element);  
        position = findElement(arr, size, element);  
        if (position != -1) {  
            printf("Element found at position: %d\n",  
position);
```

```
        } else {  
            printf("Element not found!\n");  
        }  
        break;  
case 3:  
    printf("Enter position to delete (0 to %d): ", size - 1);  
    scanf("%d", &position);  
    deleteElement(arr, &size, position);  
    break;  
case 4:  
    displayArray(arr, size);  
    break;  
case 5:  
    return 0;  
default:  
    printf("Invalid choice!\n");  
}  
  
}  
  
return 0;  
}
```

a) Insert element in an array-in single code

```
#include <stdio.h>
```

```
void insertElement(int arr[], int *size, int element, int position) {
```

```
    if (position < 0 || position > *size) {
```

```
        printf("Invalid position!\n");
```

```
        return;
```

```
    }
```

```
    for (int i = *size; i > position; i--) {
```

```
        arr[i] = arr[i - 1];
```

```
    }
```

```
    arr[position] = element;
```

```
    (*size)++;
```

```
}
```

```
void displayArray(int arr[], int size) {
```

```
    for (int i = 0; i < size; i++) {
```

```
        printf("%d ", arr[i]);
```

```
    }
```

```
    printf("\n");
```

```
}
```



```
int main() {  
    int arr[100] = {10, 20, 25, 30}; // Initialized array  
    int size = 4; // Initial size of the array  
  
    int element = 15, position = 1; // Example values  
    insertElement(arr, &size, element, position);  
    displayArray(arr, size); // Output: 10 15 20 25 30  
  
    return 0;  
}
```

b) Delete an element from the array-single code

Ans:

```
#include <stdio.h>
```

```
int main() {  
    int arr[] = {10, 20, 25, 30}; // Initialized array  
    int size = 4; // Initial size of the array  
  
    int position = 2; // Example value  
    if (position < 0 || position >= size) {  
        printf("Invalid position!\n");  
    } else {
```

```
        for (int i = position; i < size - 1; i++) {  
            arr[i] = arr[i + 1];  
        }  
        size--;  
    }  
  
    for (int i = 0; i < size; i++) {  
        printf("%d ", arr[i]);  
    }  
    printf("\n");  
  
    return 0;  
}
```

2. Matrix: 2D Array (multiplication , transpose)

(1) Ans: Multiplication

```
#include<stdio.h>
int main(){
    int i,j,k,r,c;
    printf("enter the row number: ");
    scanf("%d",&r);
    printf("enter the column number: ");
    scanf("%d",&c);
    int matrix_A[r][c],matrix_B[r][c],matrix_C[r][c];
    printf("enter the elements of matrix_A : \n");
    for ( i = 0; i < r; i++)
    {
        for ( j = 0; j < c; j++)
        {
            scanf("%d",&matrix_A[i][j]);

        }
        printf("\n");

    }
    printf("enter the elements of matrix_B : \n");
    for ( i = 0; i < r; i++)
    {
        for ( j = 0; j < c; j++)
        {
            scanf("%d",&matrix_B[i][j]);

        }
        printf("\n");

    }
```

```

    }

    for ( i = 0; i < r; i++)
    {
        for ( j = 0; j < c; j++)
        {
            matrix_C[i][j]=0;
            for ( k = 0; k < c; k++)
            {
                matrix_C[i][j]+=matrix_A[i][k]*matrix_B[k][j];
            }

        }

    }

    printf("the resultant matrix is : \n");
    for ( i = 0; i < r; i++)
    {
        for(j=0;j<c;j++){
            printf("%d\t",matrix_C[i][j]);
        }
        printf("\n");
    }

    return 0;
}

```

(2) Ans: Transpose

```
#include<stdio.h>
int main(){
    int i,j,r,c;
    printf("enter the row number: ");
    scanf("%d",&r);
    printf("enter the column number: ");
    scanf("%d",&c);
    int matrix[r][c],matrix_T[c][r];
    printf("enter the elements of matrix : \n");
    for ( i = 0; i < r; i++)
    {
        for ( j = 0; j < c; j++)
        {
            scanf("%d",&matrix[i][j]);

        }
        printf("\n");
    }

    for ( i = 0; i < r; i++)
    {
        for ( j = 0; j < c; j++)
        {
            matrix_T[j][i]= matrix[i][j];

        }
    }
}
```

```

    }

    printf("the transpose matrix is : \n");
    for ( i = 0; i < c; i++)
    {
        for(j=0;j<r;j++){
            printf("%d\t",matrix_T[i][j]);
        }
        printf("\n");
    }

    return 0;
}

```

- **LinkedList**

3. Linkedlist implementation:

Ans:

```

#include<stdio.h>
#include<stdlib.h>
typedef struct node
{
    int data;
    struct node* next;

}node;

void append(node**,int item);

```

```
void insertFront(node**,int item);  
void deleteKey(node**,int item);  
void display(node**);
```

```
int main(){  
    int ch,item,key;  
    node*head=NULL;  
    while (1)  
    {  
        printf("enter your choice\n 1.Append\n 2.InsertFront\n  
3.deleteKey\n 4.display\n 5.exit\n");  
        scanf("%d",&ch);  
  
        switch (ch)  
        {  
            case 1:printf("enter data to be inserted : \n");  
                scanf("%d",&item);  
                append(&head,item);  
                break;  
            case 2:printf("enter data to be inserted : \n");  
                scanf("%d",&item);  
                insertFront(&head,item);  
                break;  
            case 3:printf("enter data to be deleted : \n");  
                scanf("%d",&item);  
                deleteKey(&head,item);  
                break;  
            case 4:display(&head);  
                break;
```

```
case 5:exit(0);

default:printf("invalid number");
    break;
}
}

return 0;
}

void append(node**p,int item){
    node*temp=*p,*r;
    r=(node*)malloc(sizeof(node));
    r->data=item;
    r->next=NULL;
    if(*p==NULL){
        *p=r;
    }
    else{
        temp=*p;

        while (temp->next!=NULL)
        {
            temp=temp->next;
        }
        temp->next=r;
    }
}
```



```
void insertFront(node**p,int item){
    node*r;
    r=(node*)malloc(sizeof(node));
    r->data=item;
    r->next=*p;
    *p=r;
}
```

```
void deleteKey(node** p, int item) {
    node *temp = *p, *r = NULL;
    while (temp != NULL) {
        if (temp->data == item) {
            if (temp == *p) {
                *p = temp->next;
                free(temp);
                temp = *p;
            } else {
                r->next = temp->next;
                free(temp);
                temp = r->next;
            }
        } else {
            r = temp;
            temp = temp->next;
        }
    }
}
```

```
void display(node**p){
```

```

node*temp;
temp=*p;
while (temp!=NULL)
{
    printf(" %d ",temp->data);
    printf("->");
    temp=temp->next;
}
printf("NULL");
printf("\n");

}

```

- STACK

4. Stack USING ARRAY

Ans :

```
#include<stdio.h>
```

```
#define SIZE 10
```

```
typedef struct stack {
    int top;
    int item[SIZE];
} stack;
```

```
void push(stack *s, int elem) {
    if (s->top == SIZE - 1) {
        printf("Stack overflowed\n");
    }
}
```

```
    } else {  
        s->top++;  
        s->item[s->top] = elem;  
    }  
}  
  
int pop(stack *s) {  
    int element;  
    if (s->top == -1) {  
        printf("Stack underflowed\n");  
        return -1;  
    } else {  
        element = s->item[s->top];  
        s->top--;  
        return element;  
    }  
}
```

```
void display(stack *s) {  
    if (s->top == -1) {  
        printf("Stack is empty\n");  
    } else {  
        for (int i = s->top; i >= 0; i--) {  
            printf("%d ", s->item[i]);  
        }  
        printf("\n");  
    }  
}
```

```
int peep(stack *s) {  
    if (s->top == -1) {  
        printf("Stack is empty\n");  
        return -1;  
    } else {  
        return s->item[s->top];  
    }  
}
```

```
int main() {  
    stack s;  
    s.top = -1;  
    int choice, elem, result;  
  
    while (1) {  
        printf("Enter your choice:\n1. Push\n2. Pop\n3.  
Display\n4. Peep\n5. Exit\n");  
        printf("your choice is : ");  
        scanf("%d", &choice);  
  
        switch (choice) {  
            case 1:  
                printf("Enter the element to be pushed: ");  
                scanf("%d", &elem);  
                push(&s, elem);  
                break;  
  
            case 2:  
                result = pop(&s);
```

```

        if (result != -1) {
            printf("Popped element: %d\n", result);
        }
        break;

    case 3:
        display(&s);
        break;

    case 4:
        result = peep(&s);
        if (result != -1) {
            printf("Top element: %d\n", result);
        }
        break;

    case 5:
        printf("Exiting...\n");
        return 0;

    default:
        printf("Invalid choice\n");
    }
}
}
}

```

5. STACK USING LINKED LIST

Ans :

```
#include<stdio.h>
#include<stdlib.h>

typedef struct stack {
    int data;
    struct stack *next;
} stack;

void push(stack **p, int item) {
    stack *r;
    r = (stack *)malloc(sizeof(stack));
    if (r == NULL) {
        printf("Stack overflow\n");
    } else {
        r->data = item;
        r->next = *p;
        *p = r;
    }
}

int pop(stack **p) {
    stack *temp;
    int item;

    if (*p == NULL) {
        printf("Stack underflow\n");
        return -1;
    } else {
        temp = *p;
```

```
        item = temp->data;
        *p = temp->next;
        free(temp);
        return item;
    }
}

void display(stack **p) {
    stack *temp;
    temp = *p;
    if (temp == NULL) {
        printf("Stack is empty\n");
    } else {
        printf("Stack elements: ");
        while (temp != NULL) {
            printf("%d ", temp->data);
            temp = temp->next;
        }
        printf("\n");
    }
}
```

```
int peek(stack **p) {
    if (*p == NULL) {
        printf("Stack is empty\n");
        return -1;
    } else {
        return (*p)->data;
    }
}
```

```
int main() {
    stack *head = NULL;
    int choice, value;

    while(1) {
        printf("\n--- Stack Menu ---\n");
        printf("1. Push\n");
        printf("2. Pop\n");
        printf("3. Peek\n");
        printf("4. Display\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter the value to push: ");
                scanf("%d", &value);
                push(&head, value);
                break;
            case 2:
                value = pop(&head);
                if (value != -1) {
                    printf("Popped value: %d\n", value);
                }
                break;
            case 3:
                value = peek(&head);
```



```

        if (value != -1) {
            printf("Top element: %d\n", value);
        }
        break;
    case 4:
        display(&head);
        break;
    case 5:
        exit(0);
    default:
        printf("Invalid choice, please try again.\n");
    }
}

return 0;
}

```

- Stack Application

6. Infix to postfix using stack:

Ans :

```

#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>

#define MAX 100

```

```
char stack[MAX];
```

```
int top = -1;
```

```
// Function to push elements onto the stack
```

```
void push(char x) {
```

```
    if (top == MAX - 1) {
```

```
        printf("Stack overflow\n");
```

```
    } else {
```

```
        stack[++top] = x;
```

```
    }
```

```
}
```

```
// Function to pop elements from the stack
```

```
char pop() {
```

```
    if (top == -1) {
```

```
        printf("Stack underflow\n");
```

```
        return -1;
```

```
    } else {
```

```
        return stack[top--];
```

```
    }
```

```
}
```

```
// Function to return precedence of operators
```

```
int precedence(char x) {
```

```
    if (x == '+' || x == '-') {
```

```
        return 1;
```

```
    }
```

```
    if (x == '*' || x == '/') {
```

```
        return 2;
```

```

    }
    if (x == '^') {
        return 3;
    }
    return 0;
}

```

// Function to check the associativity of operators

```

int isRightAssociative(char x) {
    if (x == '^') {
        return 1; // Right associative
    }
    return 0; // Left associative for +, -, *, /
}

```

// Function to convert infix expression to postfix

```

void infixToPostfix(char *exp) {
    int i = 0, k = 0;
    char postfix[MAX];

    while (exp[i] != '\0') {
        // If the character is an operand, add it to the postfix expression
        if (isalnum(exp[i])) {
            postfix[k++] = exp[i];
        }
        // If the character is '(', push it to the stack
        else if (exp[i] == '(') {
            push(exp[i]);

```

```

    }
    // If the character is ')', pop and output from the stack
    until '(' is encountered
    else if (exp[i] == ')') {
        while (top != -1 && stack[top] != '(') {
            postfix[k++] = pop();
        }
        pop(); // Remove '(' from the stack
    }
    // If the character is an operator
    else {
        while (top != -1 && precedence(stack[top]) >
precedence(exp[i])) {
            postfix[k++] = pop();
        }
        // Handle left associativity of operators
        while (top != -1 && precedence(stack[top]) ==
precedence(exp[i]) && !isRightAssociative(exp[i])) {
            postfix[k++] = pop();
        }
        push(exp[i]);
    }
    i++;
}

// Pop all the operators from the stack
while (top != -1) {
    postfix[k++] = pop();
}

```

```

    postfix[k] = '\0'; // Null-terminate the postfix expression

    printf("Postfix expression: %s\n", postfix);
}

int main() {
    char exp[MAX];

    printf("Enter infix expression: ");
    scanf("%s", exp);

    infixToPostfix(exp);

    return 0;
}

```

.....

7. A. Postfix Evaluation----Single digit

Ans :

```

#include <stdio.h>
#include <ctype.h> // for isdigit()
#include <stdlib.h>

#define MAX 100

// Stack structure and functions
typedef struct Stack {
    int arr[MAX];

```

```
    int top;
} Stack;

void init(Stack* s) {
    s->top = -1;
}

int isEmpty(Stack* s) {
    return s->top == -1;
}

int isFull(Stack* s) {
    return s->top == MAX - 1;
}

void push(Stack* s, int value) {
    if (isFull(s)) {
        printf("Stack overflow\n");
        return;
    }
    s->arr[++(s->top)] = value;
}

int pop(Stack* s) {
    if (isEmpty(s)) {
        printf("Stack underflow\n");
        return -1;
    }
    return s->arr[(s->top)--];
}
```

```
}
```

```
// Function to evaluate postfix expression
```

```
int evaluatePostfix(char* exp) {
```

```
    Stack s;
```

```
    init(&s);
```

```
    int i = 0;
```

```
    while (exp[i] != '\0') {
```

```
        char ch = exp[i];
```

```
// Step 4: If operand, push to stack
```

```
    if (isdigit(ch)) {
```

```
        push(&s, ch - '0'); // Converting char digit to integer
```

```
    }
```

```
// Step 5: If operator, pop elements and apply operator
```

```
    else {
```

```
        int operand2 = pop(&s);
```

```
        int operand1 = pop(&s);
```

```
        switch (ch) {
```

```
            case '+':
```

```
                push(&s, operand1 + operand2);
```

```
                break;
```

```
            case '-':
```

```
                push(&s, operand1 - operand2);
```

```
                break;
```

```
            case '*':
```

```
                push(&s, operand1 * operand2);
```

```

        break;
    case '/':
        if (operand2 != 0)
            push(&s, operand1 / operand2);
        else {
            printf("Division by zero error\n");
            return -1;
        }
        break;
    default:
        printf("Unknown operator %c\n", ch);
        return -1;
    }
}
i++;
}

```

// Step 6: The final result is in the stack

```

return pop(&s);
}

```

```

int main() {
    char exp[MAX];

    printf("Enter a postfix expression (single digits and
operators): ");
    scanf("%s", exp);

    int result = evaluatePostfix(exp);
}

```



```
    if (result != -1) {  
        printf("The result of the postfix evaluation is: %d\n",  
result);  
    }  
  
    return 0;  
}
```

- QUEUE:

8. Queue Data Structure Implementation using array

Ans :

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define SIZE 100 // Maximum size of the queue
```

```
typedef struct Queue {
```

```
    int items[SIZE];
```

```
    int front;
```

```
    int rear;
```

```
} Queue;
```

```
void enqueue(Queue* q, int item);
```

```
int dequeue(Queue* q);
```

```
void display(Queue* q);
```

```
int main() {
```

```
    Queue q;
```

```
    q.front = -1;
```

```
    q.rear = -1;
```

```
    int ch, item;
```

```
    while (1) {
```

```
        printf("Enter your choice\n 1.Enqueue\n 2.Dequeue\n 3.Display\n 4.Exit\n");
```

```
        printf("Enter your choice: ");
```

```
        scanf("%d", &ch);
```

```
        switch (ch) {
```

```
            case 1:
```

```
                printf("Enter data to be inserted: \n");
```

```
                scanf("%d", &item);
```

```
                enqueue(&q, item);
```

```
        break;
    case 2:
        item = dequeue(&q);
        if (item != -1)
            printf("Dequeued item: %d\n", item);
        break;
    case 3:
        display(&q);
        break;
    case 4:
        exit(0);
    default:
        printf("Invalid choice\n");
        break;
    }
}

return 0;
}
```

```
void enqueue(Queue* q, int item) {
    if (q->rear == SIZE - 1) {
```

```
    printf("Queue is full\n");  
    return;  
}  
if (q->front == -1) // If queue is empty  
    q->front = 0;  
q->rear++;  
q->items[q->rear] = item;  
printf("Inserted %d\n", item);  
}
```

```
int dequeue(Queue* q) {  
    int item;  
    if (q->front == -1) {  
        printf("Queue is empty\n");  
        return -1;  
    }  
    item = q->items[q->front];  
    q->front++;  
    if (q->front > q->rear) { // Queue becomes empty after  
dequeuing  
        q->front = q->rear = -1;  
    }  
}
```

```
    return item;
}

void display(Queue* q) {
    if (q->rear == -1) {
        printf("Queue is empty\n");
        return;
    }
    printf("Queue elements: \n");
    for (int i = q->front; i <= q->rear; i++) {
        printf("%d ", q->items[i]);
    }
    printf("\n");
}
```

9. Queue Data Structure Using circular array:

Ans :

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define SIZE 100 // Maximum size of the queue
```

```
typedef struct Queue {
```

```
int items[SIZE];
int front;
int rear;
} Queue;

void enqueue(Queue* q, int item);
int dequeue(Queue* q);
void display(Queue* q);

int main() {
    Queue q;
    q.front = -1;
    q.rear = -1;

    int ch, item;
    while (1) {
        printf("Enter your choice\n 1.Enqueue\n 2.Dequeue\n
3.Display\n 4.Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &ch);

        switch (ch) {
            case 1:
                printf("Enter data to be inserted: \n");
                scanf("%d", &item);
                enqueue(&q, item);
                break;
            case 2:
                item = dequeue(&q);
```

```
        if (item != -1)
            printf("Dequeued item: %d\n", item);
        break;
    case 3:
        display(&q);
        break;
    case 4:
        exit(0);
    default:
        printf("Invalid choice\n");
        break;
    }
}

return 0;
}
```

```
void enqueue(Queue* q, int item) {
    if ((q->rear + 1) % SIZE == q->front) {
        printf("Queue is full\n");
        return;
    }
    if (q->front == -1) {
        q->front = 0;
    }
    q->rear = (q->rear + 1) % SIZE;
    q->items[q->rear] = item;
    printf("Inserted %d\n", item);
}
```

```
int dequeue(Queue* q) {
    int item;
    if (q->front == -1) {
        printf("Queue is empty\n");
        return -1;
    }
    item = q->items[q->front];
    if (q->front == q->rear) {
        q->front = q->rear = -1;
    } else {
        q->front = (q->front + 1) % SIZE;
    }
    return item;
}
```

```
void display(Queue* q) {
    if (q->front == -1) {
        printf("Queue is empty\n");
        return;
    }
    printf("Queue elements: \n");
    int i = q->front;
    while (1) {
        printf("%d ", q->items[i]);
        if (i == q->rear) break;
        i = (i + 1) % SIZE;
    }
    printf("\n");
}
```



```
}
```

10. Queue using LinkedList:

Ans :

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
typedef struct node {  
    int data;  
    struct node* next;  
} node;
```

```
typedef struct Queue {  
    node* front;  
    node* rear;  
} Queue;
```

```
void enqueue(Queue*, int);  
int dequeue(Queue*);  
void display(Queue*);
```

```
int main() {  
    int ch, item;  
    Queue q;  
    q.front = q.rear = NULL;  
  
    while (1) {
```

```
printf("Enter your choice\n 1. Enqueue\n 2. Dequeue\n  
3. Display\n 4. Exit\n");  
printf("Enter your choice: ");  
scanf("%d", &ch);
```

```
switch (ch) {  
    case 1:  
        printf("Enter data to be inserted: ");  
        scanf("%d", &item);  
        enqueue(&q, item);  
        break;  
    case 2:  
        item = dequeue(&q);  
        if (item != -1) {  
            printf("Dequeued item: %d\n", item);  
        }  
        break;  
    case 3:  
        display(&q);  
        break;  
    case 4:  
        exit(0);  
    default:  
        printf("Invalid choice!\n");  
        break;  
}  
}
```

```
return 0;
```

```
}
```

```
void enqueue(Queue* q, int item) {  
    node* newNode = (node*)malloc(sizeof(node));  
    if (!newNode) {  
        printf("Memory allocation failed\n");  
        return;  
    }  
    newNode->data = item;  
    newNode->next = NULL;  
  
    if (q->rear == NULL) { // Empty queue  
        q->front = q->rear = newNode;  
    } else {  
        q->rear->next = newNode;  
        q->rear = newNode;  
    }  
    printf("Enqueued: %d\n", item);  
}
```

```
int dequeue(Queue* q) {  
    if (q->front == NULL) {  
        printf("Queue is empty\n");  
        return -1;  
    }  
    node* temp = q->front;  
    int item = temp->data;  
    q->front = q->front->next;
```

```

    if (q->front == NULL) { // If the queue becomes empty
        q->rear = NULL;
    }
    free(temp);
    return item;
}

```

```

void display(Queue* q) {
    if (q->front == NULL) {
        printf("Queue is empty\n");
        return;
    }
    node* temp = q->front;
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

```

11. Sorting Algorithms:

a. Linear search

Ans :

```
#include<stdio.h>
```

```

void linearSearch(int arr[], int n, int key) {
    int found = 0;
    for (int i = 0; i < n; i++) {

```

```
        if (arr[i] == key) {
            printf("Element %d found at index %d\n", key, i);
            found = 1;
            break;
        }
    }
    if (!found) {
        printf("Element %d not found in the array\n", key);
    }
}
```

```
int main() {
    int n, key;

    printf("Enter the number of elements in the array: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter the elements of the array:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    printf("Enter the element to search for: ");
    scanf("%d", &key);

    linearSearch(arr, n, key);
}
```

```
    return 0;  
}
```

b. Binary Search:

Ans :

```
#include<stdio.h>
```

```
int binarySearch(int arr[], int n, int key) {  
    int low = 0, high = n - 1;
```

```
    while (low <= high) {  
        int mid = (low + high) / 2;
```

```
        if (arr[mid] == key) {  
            return mid;  
        } else if (arr[mid] < key) {  
            low = mid + 1;  
        } else {  
            high = mid - 1;  
        }  
    }
```

```
    return -1;  
}
```

```
int main() {  
    int n, key;
```

```
printf("Enter the number of elements in the array: ");
scanf("%d", &n);

int arr[n];

printf("Enter the sorted elements of the array:\n");
for (int i = 0; i < n; i++) {
    scanf("%d", &arr[i]);
}

printf("Enter the element to search for: ");
scanf("%d", &key);

int result = binarySearch(arr, n, key);

if (result != -1) {
    printf("Element %d found at index %d\n", key,
result);
} else {
    printf("Element %d not found in the array\n", key);
}

return 0;
}
```

12. Sorting Algorithm:

a. Bubble Sorting

Ans :

```
#include <stdio.h>
```

```
void bubbleSort(int a[], int n) {  
    int i, j, t, flag;  
    for (i = 0; i < n - 1; i++) {  
        flag = 0;  
        for (j = 0; j < n - 1 - i; j++) {  
            if (a[j] > a[j + 1]) {  
                t = a[j];  
                a[j] = a[j + 1];  
                a[j + 1] = t;  
                flag = 1;  
            }  
        }  
        if (flag == 0) {  
            break;  
        }  
    }  
}
```

```
int main() {  
    int n, i;  
    printf("Enter number of elements: ");
```



```
scanf("%d", &n);

int a[n];

for (i = 0; i < n; i++) {
    printf("Element at index %d is : ",i);
    scanf("%d", &a[i]);
}

bubbleSort(a, n);

printf("Sorted array: ");
for (i = 0; i < n; i++) {
    printf("%d ", a[i]);
}

return 0;
}
```

b. Selection Sorting

Ans :

```
#include <stdio.h>
```

```
void selectionSort(int a[], int n) {
```

```
    int i, j, k, t;
```

```
    for (i = 0; i < n - 1; i++) {
```

```
        k = i;
```

```
        for (j = i + 1; j < n; j++) {
```

```
            if (a[j] < a[k]) {
```

```
                k = j;
```

```
            }
```

```
        }
```

```
        if (k != i) {
```

```
            t = a[k];
```

```
            a[k] = a[i];
```

```
            a[i] = t;
```

```
        }
```

```
    }
```

```
}
```

```
int main() {
```

```
    int n, i;
```

```
    printf("Enter number of elements: ");
```

```
    scanf("%d", &n);
```

```
    int a[n];
```

```
for (i = 0; i < n; i++) {  
    printf("Elemnt at the index %d is : ",i);  
    scanf("%d", &a[i]);  
}
```

```
selectionSort(a, n);
```

```
printf("Sorted array: ");  
for (i = 0; i < n; i++) {  
    printf("%d ", a[i]);  
}
```

```
    return 0;  
}
```

c. Insertion sorting:

Ans :

```
#include <stdio.h>
```

```
#define MAX_SIZE 100
```

```
void insertionSort(int a[], int n) {  
    int i, j, key;  
    for (j = 1; j < n; j++) {  
        key = a[j];  
        i = j - 1;
```

```
        while (i >= 0 && a[i] > key) {
```

```
        a[i + 1] = a[i];  
        i = i - 1;  
    }  
    a[i + 1] = key;  
}  
}
```

```
int main() {  
    int n, i;  
    printf("Enter number of elements (<= 100): ");  
    scanf("%d", &n);
```

```
  
    int a[MAX_SIZE];
```

```
  
    for (i = 0; i < n; i++) {  
        printf("Elemnt at the position %d is : ",i);  
        scanf("%d", &a[i]);  
    }
```

```
    insertionSort(a, n);
```

```
  
    printf("Sorted array: ");  
    for (i = 0; i < n; i++) {  
        printf("%d ", a[i]);  
    }
```

```
    return 0;
```

d. Merge Sort:

Ans :

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
void merge(int arr[], int p, int q, int r) {
```

```
    int n1 = q - p + 1;
```

```
    int n2 = r - q;
```

```
    int L[n1 + 1], R[n2 + 1];
```

```
    for (int i = 0; i < n1; i++) {
```

```
        L[i] = arr[p + i];
```

```
    }
```

```
    for (int j = 0; j < n2; j++) {
```

```
        R[j] = arr[q + 1 + j];
```

```
    }
```

```
    L[n1] = INT_MAX;
```

```
    R[n2] = INT_MAX;
```

```
    int i = 0, j = 0;
```

```
    for (int k = p; k <= r; k++) {
```

```
        if (L[i] <= R[j]) {
```

```
            arr[k] = L[i];
```

```
            i++;
```

```
        } else {
```

```
        arr[k] = R[j];
        j++;
    }
}
}
```

```
void mergeSort(int arr[], int p, int r) {
    if (p < r) {
        int q = p + (r - p) / 2;
        mergeSort(arr, p, q);
        mergeSort(arr, q + 1, r);
        merge(arr, p, q, r);
    }
}
```

```
void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}
```

```
int main() {
    int arr[] = {12, 11, 13, 5, 6, 7};
    int arr_size = sizeof(arr) / sizeof(arr[0]);

    printf("Given array is: \n");
    printArray(arr, arr_size);
}
```

```
mergeSort(arr, 0, arr_size - 1);

printf("\nSorted array is: \n");
printArray(arr, arr_size);

return 0;
}
```

e. Quick Sort:

Ans :

```
#include <stdio.h>
```

```
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
```

```
int partition(int a[], int p, int r) {
    int x = a[r];
    int i = p - 1;

    for (int j = p; j <= r - 1; j++) {
        if (a[j] <= x) {
            i++;
            swap(&a[i], &a[j]);
        }
    }
    swap(&a[i + 1], &a[r]);
}
```

```
        return i + 1;
    }

void quickSort(int a[], int p, int r) {
    if (p < r) {
        int q = partition(a, p, r);
        quickSort(a, p, q - 1);
        quickSort(a, q + 1, r);
    }
}

void printArray(int a[], int size) {
    for (int i = 0; i < size; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");
}

int main() {
    int a[] = {12, 4, 13, 9, 5, 6, 10};
    int size = sizeof(a) / sizeof(a[0]);

    printf("Unsorted array: ");
    printArray(a, size);

    quickSort(a, 0, size - 1);

    printf("Sorted array: ");
    printArray(a, size);
}
```



```
    return 0;  
}
```

f. Counting Sort

Ans:

```
#include <stdio.h>  
#include <stdlib.h>  
#include <limits.h>
```

```
int maximum(int arr[], int n) {  
    int max = INT_MIN;  
    for (int i = 0; i < n; i++) {  
        if (arr[i] > max) {  
            max = arr[i];  
        }  
    }  
    return max;  
}
```

```
int countSort(int arr[], int n) {  
    int maxElem = maximum(arr, n);  
    int count[maxElem + 1];  
  
    for (int i = 0; i <= maxElem; i++) {  
        count[i] = 0;  
    }  
  
    for (int i = 0; i < n; i++) {
```

```
        count[arr[i]]++;
    }

    int i = 0, j = 0;
    while (j <= maxElem) {
        if (count[j] > 0) {
            arr[i++] = j;
            count[j]--;
        } else {
            j++;
        }
    }

    return 0;
}

int main() {
    int arr[10] = {2, 2, 1, 3, 3, 4, 4, 5, 6, 5};
    int n = 10;

    countSort(arr, n);

    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

g. Bucket Sort

Ans:

```
#include<stdio.h>
void bucketSort(int A[],int n){
    int max=A[0];
    for(int i=1;i<n;i++){
        if(A[i]>max){
            max=A[i];
        }
    }
    int bucket[max+1];
    for(int i=0;i<=max;i++){
        bucket[i]=0;
    }
    for(int i=0;i<n;i++){
        bucket[A[i]]++;
    }

    for(int i=0,j=0;i<=max;i++){
        while(bucket[i]>0){
            A[j++]=i;
            bucket[i]--;
        }
    }
}

int main(){
    printf("Enter the number of elements:");
```

```

    int n;
    scanf("%d",&n);
    int A[n];
    printf("Enter the elements:");
    for(int i=0;i<n;i++){
        scanf("%d",&A[i]);
    }
    bucketSort(A,n);
    printf("The sorted array is:");
    for(int i=0;i<n;i++){
        printf("%d ",A[i]);
    }
    return 0;
}

```

h. Radix Sort

```

i. #include <stdio.h>
j.
k. // Function to find the maximum value in the array
l. int getMax(int arr[], int n) {
m.     int max = arr[0];
n.     for (int i = 1; i < n; i++) {
o.         if (arr[i] > max) {
p.             max = arr[i];
q.         }
r.     }
s.     return max;
t. }
u.
v. // Function to perform Counting Sort on a specific digit
   (represented by exp)
w. void countingSort(int arr[], int n, int exp) {
x.     int output[n];           // Output array to store sorted
                                   numbers

```

```

y.    int count[10] = {0}; // Count array to store occurrences
    of digits
z.
aa.    // Count occurrences of each digit in the current place
    value
bb.    for (int i = 0; i < n; i++) {
cc.        int digit = (arr[i] / exp) % 10;
dd.        count[digit]++;
ee.    }
ff.
gg.    // Update the count array to store the position of each
    digit
hh.    for (int i = 1; i < 10; i++) {
ii.        count[i] += count[i - 1];
jj.    }
kk.
ll.    // Build the output array
mm.    for (int i = n - 1; i >= 0; i--) {
nn.        int digit = (arr[i] / exp) % 10;
oo.        output[count[digit] - 1] = arr[i];
pp.        count[digit]--;
qq.    }
rr.
ss.    // Copy the sorted elements back to the original array
tt.    for (int i = 0; i < n; i++) {
uu.        arr[i] = output[i];
vv.    }
ww. }
xx.
yy. // Function to perform Radix Sort on the array
zz. void radixSort(int arr[], int n) {
aaa.     int max = getMax(arr, n); // Get the maximum number
    in the array
bbb.
ccc.     // Perform Counting Sort for each digit, starting
    from the least significant digit (LSD)
ddd.     for (int exp = 1; max / exp > 0; exp *= 10) {
eee.         countingSort(arr, n, exp);
fff.     }
ggg. }
hhh.
iii.    // Function to print the array

```

```

jjj.    void printArray(int arr[], int n) {
kkk.        for (int i = 0; i < n; i++) {
lll.            printf("%d ", arr[i]);
mmm.        }
nnn.        printf("\n");
ooo.    }
ppp.
qqq.    // Driver function to test Radix Sort
rrr.    int main() {
sss.        int arr[] = {170, 45, 75, 90, 802, 24, 2, 66};
ttt.        int n = sizeof(arr) / sizeof(arr[0]);
uuu.
vvv.        printf("Original array: ");
www.        printArray(arr, n);
xxx.
yyy.        radixSort(arr, n);
zzz.
aaaa.        printf("Sorted array: ");
bbbb.        printArray(arr, n);
cccc.
dddd.        return 0;
eeee.    }
ffff.

```

13. Binary Tree Implementation:

Ans:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

// Define the structure for the binary tree node

```
typedef struct node {
```

```
    int data;
```

```
    struct node *lc; // Left child
```

```
    struct node *rc; // Right child
```

```
} node;
```

// Function to create a new node

```
node* createNode(int data) {  
    node *newNode = (node *)malloc(sizeof(node));  
    newNode->data = data;  
    newNode->lc = newNode->rc = NULL;  
    return newNode;  
}
```

// In-order traversal (Left, Root, Right)

```
void inorder(node *t) {  
    if (t) {  
        inorder(t->lc);    // Visit left subtree  
        printf("%d ", t->data); // Visit root  
        inorder(t->rc);    // Visit right subtree  
    }  
}
```

// Pre-order traversal (Root, Left, Right)

```
void preorder(node *t) {  
    if (t) {  
        printf("%d ", t->data); // Visit root  
        preorder(t->lc);    // Visit left subtree  
        preorder(t->rc);    // Visit right subtree  
    }  
}
```

// Post-order traversal (Left, Right, Root)

```
void postorder(node *t) {  
    if (t) {
```

```

        postorder(t->lc);    // Visit left subtree
        postorder(t->rc);    // Visit right subtree
        printf("%d ", t->data); // Visit root
    }
}

// Main function to test the tree and traversals
int main() {
    // Create nodes
    node *root = createNode(1);
    node *leftChild = createNode(2);
    node *rightChild = createNode(3);
    node *leftLeftChild = createNode(4);
    node *leftRightChild = createNode(5);

    // Build the tree
    root->lc = leftChild;
    root->rc = rightChild;
    leftChild->lc = leftLeftChild;
    leftChild->rc = leftRightChild;

    // Print the traversals
    printf("In-order traversal: ");
    inorder(root);
    printf("\n");

    printf("Pre-order traversal: ");
    preorder(root);
    printf("\n");
}

```



```

printf("Post-order traversal: ");
postorder(root);
printf("\n");

return 0;
}

```

```

// BinaryTree.h, if ($?) { .\BinaryTree }
In-order traversal: 4 2 5 1 3
Pre-order traversal: 1 2 4 5 3
Post-order traversal: 4 5 2 3 1
PS C:\Users\skari\DSA C>

```

1. Stack

- **LIFO:** Last In First Out
- **Operations:**
 - **Push:** Add element to the top
 - **Pop:** Remove element from the top
- **Key Terminology:**
 - **Top:** The most recent element added (the one at the top)
- **Common Methods:**
 - push(), pop(), peek(), isEmpty()

2. Queue

- **FIFO:** First In First Out
- **Operations:**

- **Enqueue:** Add element to the rear
- **Dequeue:** Remove element from the front
- **Key Terminology:**
 - **Front:** The element that will be removed next
 - **Rear:** The element that was most recently added
- **Common Methods:**
 - enqueue(), dequeue(), isEmpty(), peek()

3. Linked List

- **Singly Linked List:** Each node points to the next node.
 - **Head:** The starting node
 - **Tail:** The last node (points to null)
- **Doubly Linked List:** Nodes point to both next and previous nodes.
 - **Head:** Starting node
 - **Tail:** The last node
- **Circular Linked List:** The last node points back to the first node.
- **Operations:**
 - insertFirst(), insertLast(), deleteFirst(), deleteLast(), search()

4. Priority Queue

- **Operations:**
 - **Enqueue:** Insert element based on priority
 - **Dequeue:** Remove element with the highest priority
- **Key Terminology:**
 - **Front:** The highest-priority element
 - **Rear:** The lowest-priority element

5. Deque (Double-Ended Queue)

- **Operations:**
 - **Enqueue Front:** Add element to the front
 - **Enqueue Rear:** Add element to the rear
 - **Dequeue Front:** Remove element from the front
 - **Dequeue Rear:** Remove element from the rear
- **Key Terminology:**
 - **Front:** The element at the front

- **Rear:** The element at the rear

6. Tree

- **Nodes:** Elements in the tree
- **Root:** The topmost node
- **Leaf:** A node without children
- **Subtree:** A tree formed by a node and its descendants
- **Binary Tree:** Each node has at most two children (left, right)

7. Graph

- **Vertices (Nodes):** Points in the graph
- **Edges:** Connections between the vertices
- **Directed/Undirected:** Whether edges have directions or not
- **Adjacency List/Matrix:** Representation methods for graphs

8. Hash Table

- **Key:** Unique identifier for data
- **Value:** Data associated with the key
- **Hash Function:** Maps keys to indices
- **Collision:** When two keys hash to the same index
